Poison-Resilient Anomaly Detection: Mitigating Poisoning Attacks in Semi-Supervised Encrypted Traffic Anomaly Detection

Zhangfa Wu, Huifang Li, Yekui Qian, Yi Hua, and Hongping Gan, *Member, IEEE*

***Abstract*—Semi-supervised encrypted traffic anomaly detection models in zero-positive scenarios are susceptible to human labeling errors or poisoning attacks, thereby compromising the stability and reliability of the model. However, existing methods are insufficient to address the challenge of reduced inter-class distance caused by poisoning attacks and the inability of reconstruction error to serve as a reliable detection criterion. To alleviate these challenges, a framework called Poison-Resistant Anomaly Detection (PRAD) is proposed to mitigate poisoning attacks and enhance anomaly detection performance. Specifically, a feature encoding module autoencoder-based is first designed that simultaneously leverages the Amsgrad gradient descent algorithm and the warm-up strategy to enhance the feature extraction and generalization capabilities, thereby alleviating the reduction of inter-class distance. Additionally, a feature analysis module is introduced to measure the impact of poisoning attacks on inter-class distance and the distribution of reconstruction errors, which provides valuable prior information for subsequent anomaly detection tasks. Finally, an online clustering-based anomaly detection algorithm that utilizes the extracted features and their corresponding reconstruction errors are developed to address the issue of detection criteria. Experimental results on public benchmark datasets demonstrate that PRAD exhibits significantly superior poison-resilient capabilities compared to other semi-supervised anomaly detection methods in anomaly detection tasks under poisoning attacks.**

***Index Terms*—Encrypted traffic, online clustering, poisoning attack defense, semi-supervised anomaly detection.**

I. Introduction

THE encrypted communication protocols, while safeguarding user data privacy [1], pose challenges to conventional anomaly detection techniques that operate on plaintext data [2], [3]. To detect anomalous behaviors in encrypted communications, researchers employ diverse anomaly detection methods to identify data that exhibit significant deviations from the majority of samples. These approaches help mitigate potential risks and abnormal behaviors in communication networks, ensuring the security of communication systems and safeguarding user privacy. Given the scarcity of abnormal data in real-world scenarios and the difficulty in obtaining labeled samples due to their concealment by normal data points. Semi-supervised learning algorithms have emerged as a promising approach, which require only a limited amount of labeled data and reduced labor costs, offering a solid theoretical foundation for anomaly detection in encrypted traffic.

Semi-supervised anomaly detection methods can be categorized into traditional anomaly detection methods and Deep Learning (DL) methods. Traditional anomaly detection methods typically begin by modeling the statistical characteristics of normal data and subsequently identify anomalous data points by detecting those that deviate significantly from this model. Examples of such methods include the density estimation method LOF [4], the clustering method OC-SVM [5], probability-based estimation methods such as ECOD [6], COPOD [7], etc., as well as ensemble learning anomaly detection methods such as IForest [8], SUOD [9], LSCP [10], etc. In practical applications, traditional anomaly detection methods often face challenges in capturing intricate data patterns and sample dependencies, and are highly sensitive to noisy data. Consequently, researchers gradually shifted their focus to DL-based anomaly detection methods, which typically employ deep neural networks to automatically learn complex feature representations of data, enabling the identification of abnormal data. In comparison to traditional anomaly detection methods, DL-based methods excel in effectively capturing data patterns and sample features, leading to enhanced anomaly detection performance.

DL-based anomaly detection methods can be categorized into two types based on their training approaches, i.e., Based on End-to-End learning (E2E) methods and based on Representation Learning (RL) methods. The E2E-based methods entail directly mapping input data to output results, bypassing intermediate processing stages like feature extraction and data transformation [11]. In real-world scenarios, E2E-based methods frequently face significant susceptibility to noise or contaminated data present in the training set. This susceptibility results in the

Manuscript received 13 January 2024; revised 7 April 2024; accepted 29 April 2024. Date of publication 10 May 2024; date of current version 16 August 2024. This work was supported in part by the Innovation Workstation of State Key Laboratory of Intelligent Gaming under Grant ZBKF-23-03, and in part by the Basic Research Programs of Taicang under Grant TC2022JC19. Recommended for acceptance by Dr. Y. Wu. *(Corresponding author: Huifang Li.)*

Zhangfa Wu and Huifang Li are with the School of Electronic Information, Northwestern Polytechnical University, Xi'an 710129, China (e-mail: zhangfa_wu2021100697@mail.nwpu.edu.cn; Lhuifang@nwpu.edu.cn).

Yekui Qian is with People's Liberation Army Air Force Academy, Zhengzhou 450052, China (e-mail: qyk1129@163.com).

Yi Hua is with the School of Aeronautics, Northwestern Polytechnical University, Xi'an 710072, China (e-mail: yihua@mail.nwpu.edu.cn).

Hongping Gan is with the School of Software, Northwestern Polytechnical University, Xi'an 710129, China (e-mail: ganhongping@nwpu.edu.cn).

Digital Object Identifier 10.1109/TNSE.2024.3397719

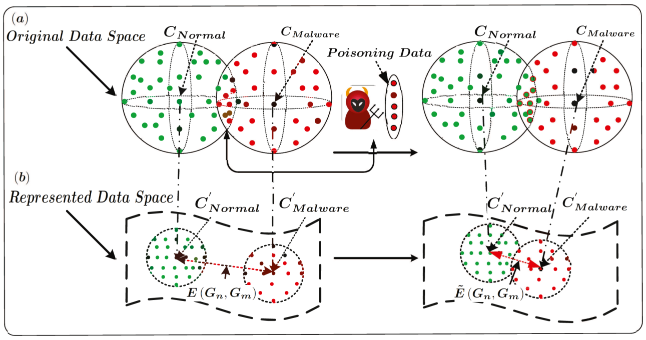

Fig. 1. Change trends of representation features under normal training and poisoning attacks. (a) represents the data distribution in the original data space and the process of filtering and injecting poisoned samples. (b) represents the change of spatial features in the hidden layer. The mathematical symbols $\mathcal{C}$ and $\mathcal{C}'$ represent the class centers of the original space and the hidden layer feature space, respectively. The mathematical symbols $E(G_n, G_m)$ and $\tilde{E}(G_n, G_m)$ represent the inter-class distance under normal training and the inter-class distance under poisoning attack, respectively.

model overfitting to the training data and subsequently exhibiting subpar performance on the test set. Moreover, a notable limitation of these methods is their lack of interpretability. RL-based anomaly detection methods offer several advantages, including effective dimensionality reduction, efficient data storage, and compatibility with conventional anomaly detection algorithms. RL-based anomaly detection methods can additionally provide interpretability, rendering them valuable in real-world applications. Nevertheless, the presence of human labeling errors or data poisoning attacks significantly restricts the applicability of RL-based semi-supervised anomaly detection algorithms [12]. In summary, the E2E-based methods offer a direct mapping approach but are susceptible to issues like overfitting and data poisoning attacks. Conversely, RL-based methods provide the advantages of dimensionality reduction and interpretability but are also limited by the impact of human labeling errors and data poisoning attacks in real-world scenarios.

To address the limitations of the aforementioned methods, researchers have devised and implemented various improvements and techniques aimed at mitigating poisoning attacks in anomaly detection problems. For instance, Bovenzia et al. [13] proposed an enhanced version of KitNet to address the issue of poisoning attacks in E2E-based learning. This enhancement aims to mitigate the impact of poisoned data on the E2E-based learning process. Regarding semi-supervised anomaly detection problems RL-based, researchers often adopt poisoning defense strategies inspired by traditional anomaly detection algorithms [14], [15]. These defense strategies employ the proportion of outliers in the dataset to define the threshold of the decision function in the fitting process. However, these strategies often prove inadequate when confronted with complex attack scenarios, particularly when adversaries construct poisoning attack samples with high similarity to normal samples. Fig. 1 depicts the changes in hidden layer features and reconstruction errors during normal training and under poisoning attacks. In summary, researchers have made some progress to address the shortcomings of existing methods, but the following challenges still remain:

- Poisoning attacks substantially reduce the inter-class distance, hindering the model's ability to accurately learn and represent normal traffic data. Consequently, this severely degrades the performance of anomaly detection.
- Meticulously crafted poisoned samples exhibit similar distributions to normal samples, rendering the reconstruction error ineffective as a classification criterion for anomaly detection.

A. Motivation and Contributions

In this work, a framework called Poison-Resistant Anomaly Detection (PRAD) is proposed to address the challenge of detecting anomalies in encrypted traffic, which may arise from poisoning attacks and lead to diminished inter-class distance and unavailable reconstruction errors. PRAD can effectively alleviate poisoning attacks and improve anomaly detection performance. Specifically, we first design a feature encoding module based on a variant of Variational Recurrent Neural Network (VRNN) to address the inherent curse of dimensionality in high-dimensional traffic data. The Amsgrad gradient descent algorithm and warm-up strategy are employed to further enhance PRAD's feature extraction and generalization capabilities, thereby mitigating the reduction of inter-class distance. Furthermore, a feature analysis module is designed to quantify the impact of poisoning attacks on inter-class distance and reconstruction error distribution, which can provide valuable prior information for subsequent anomaly detection tasks. Finally, we design an anomaly detection algorithm that employs online clustering, which leverages the extracted features and their associated reconstruction errors. This proposed algorithm performs clustering on the extracted features using a Gaussian Mixture Model (GMM) and assigns labels to distinct clusters based on the corresponding reconstruction errors.

The main contributions of this paper can be summarized as follows:

- We propose a PRAD framework for detecting encrypted traffic anomalies in semi-supervised poisoning attack scenarios, which effectively leverages the interdependencies between labeled samples and data priors for mitigating poisoning attacks, leading to superior anomaly detection performance.
- We introduce a feature extraction module that utilizes VRNN-based feature encoding, and subsequently utilize the Amsgrad gradient descent algorithm combined with a warm-up strategy to alleviate the reduction of inter-class distance.
- We propose feature-based inter-class distance similarity and intra-class reconstruction error distribution-based shift evaluation for poisoning attack impact estimation.
- We design an online clustering algorithm to efficiently detect anomalies in encrypted traffic, which leverages the extracted features and their associated reconstruction errors. By clustering the extracted features using a GMM and assigning labels to different clusters based on the reconstruction errors, anomalies in the encrypted traffic can be efficiently identified.

Experimental results demonstrate that PRAD outperforms other RL-based semi-supervised anomaly detection algorithms in terms of AUCROC and AUCPR scores on both the CTU13 and CIRA-CIC-DoHBrw-2020 datasets.

B. Organization

The remainder of this paper is organized as follows. Section II briefly reviews related work. Section III describes the threat model and defense capabilities. Section IV presents the details of the proposed PRAD framework. Section V compares the test results of the PRAD framework and ablation experiments. Finally, Section VI concludes the paper.

II. Related Works

A. Semi-Supervised Traffic Anomaly Detection Algorithm

Given the limited availability of labeled abnormal traffic data, most large-scale traffic anomaly detection tasks are carried out under zero-positive settings. Traffic camouflage techniques (such as traffic shaping and link padding) and the "curse of dimensionality" resulting from the inherently high dimensionality of traffic data can impede the training and prediction capabilities of conventional models. DL-based anomaly detection methods can effectively address this challenge and substantially enhance the performance of anomaly detection in encrypted traffic scenarios. Most E2E-based traffic anomaly detection methods leverage neural networks to extract local or global features from traffic data. These features are then used in scoring mechanisms or sample reconstruction errors to evaluate test samples and identify anomalous data [11], [16], [17]. RL-based traffic anomaly detection algorithms also employ neural networks to embed the original data into low dimensions using manifold learning techniques. This low-dimensional feature representation is then utilized in conjunction with traditional anomaly detection algorithms for anomaly detection tasks [6], [7], [9], [10]. For example, Cao et al. [12] proposed an anomaly detection algorithm that combines deep features extracted from Autoencoders (AEs) with traditional one-class classification. This approach leverages the powerful feature learning capabilities of DL and the concept of manifold learning to transform the original high-dimensional data into a low-dimensional space. Subsequently, anomalies are classified using an anomaly detection algorithm without altering the original data distribution.

B. Semi-Supervised Anomaly Detection Poisoning Attacks and Defense

1) Semi-Supervised Poisoning Attack: Poisoning attacks, as proposed by Barreno et al. [18], pose a threat to the training phase of Machine Learning (ML) models. Adversaries aim to compromise the integrity and reliability of the classifier by poisoning the training data. Poisoning attacks employ various techniques, such as modifying labels, injecting malicious samples, and manipulating data. These attacks can be categorized into two types: untargeted attacks, which aim to degrade a model's overall performance [19], [20], and targeted attacks, which aim to manipulate a model's predictions for specific outcomes [21], [22], [23]. With respect to the construction of poisoning samples, attacks can be categorized into label manipulation and data manipulation. Data manipulation attacks include optimization-based methods [24], [25] and training-based methods [26], [27], [28], [29]. The integrity and auditability requirements imposed by network protocols on encrypted traffic data pose significant challenges to anomaly detection in poisoning attacks based on training, necessitating the predominant use of optimization-based methods. Bovenzi et al. [30] demonstrated the impact of poisoning attacks, specifically using label flipping, on semi-supervised anomaly detection models.

2) Semi-Supervised Poisoning Attack Defense: Data poisoning defense can be broadly categorized into traditional data poisoning defense methods and DL-based data poisoning defense methods. Traditional data poisoning defense methods emphasize preprocessing the data before model training to identify and eliminate anomalies, exemplified by data cleaning techniques employing anomaly detectors [14], [15]. In contrast, DL-based data poisoning defense methods typically involve preprocessing the dataset before training, such as leveraging knowledge extracted from a small set of clean data to bolster robustness [31]. Optimization-based methods enhance the model's resilience [31], [32], [33], including dynamic model training [32] and loss correction using a trusted small dataset [31]. Within the context of Android malware detection, label-based semi-supervised defense, cluster-based semi-supervised defense [34], and deep K-NN [33] defense have been proposed as effective countermeasures to detect and eliminate poisoned samples.

Poisoning attack defense methods are designed to mitigate the impact of poisoning attacks on anomaly detection algorithms. However, traditional poisoning defense methods may be insufficient in eliminating similar poisoned samples, while deep learning-based poisoning defense methods, while dependent on affected training samples, cannot ensure the complete eradication of poisoned data. Existing methods also struggle due to limited data availability and similarities in poisoned data patterns. The proposed PRAD framework adopts feature encoding to enhance feature extraction and alleviate the reduction of inter-class distance. An online clustering algorithm combined with reconstruction loss is used to effectively detect anomalies. PRAD demonstrates superior performance compared to other methods in mitigating attacks and achieving superior anomaly detection performance.

III. Threat Model and Defense Capability

In this section, we present a threat model for the poisoning attack and analyze the advantages it provides to defenders. The threat model includes the adversary's goal, knowledge, and capabilities in manipulating training data, as detailed in Section III-A. The strategies employed by defenders to effectively prevent poisoning attacks are significantly influenced by the information obtained from these threat models, as discussed in Section III-B.

A. Goal, Knowledge and Capability of Adversary

The adversary's goal is to undermine the model's performance. Their level of knowledge regarding the target ML model can vary. In a white-box attack [35], they possess direct access to training parameters, while in a black-box attack [36], they can obtain similar data through manipulation techniques. Consider a victim training an anomaly detection algorithm on a limited dataset, an adversary who can upload data to the Internet can manipulate parts of the unlabeled dataset. To ensure data integrity and auditability, adversaries may use label-flipping attacks and optimization-based data manipulation.

Formally, poisoning adversary $\mathcal{A}$ in the unlabeled dataset constructs a set of poisoning examples $\mathcal{S}_\mathcal{P}$, the input $\boldsymbol{x}^*$ represents the adversary's carefully crafted poisoning samples injected into the training set, $y^* \neq c(\boldsymbol{x}^*)$ represents the desired false target label, $\mathcal{N}_\mathcal{P}$ denotes the maximum number of poisoning samples that the adversary can inject, the neural network model is denoted by f, the training algorithm is represented by $\mathcal{T}_s$ and $\boldsymbol{x}$ is a labeled subset of the original dataset $\mathcal{X}$, there we have:

$$\mathcal{S}_\mathcal{P} \leftarrow \mathcal{A}\left(\boldsymbol{x}^*, y^*, \mathcal{N}_\mathcal{P}, f, \mathcal{T}_s, \boldsymbol{x}\right). \tag{1}$$

The adversary's goal is to manipulate the victim's model such that the resulting model $f \leftarrow \mathcal{T}s(\mathcal{S}_\mathcal{P} \cup \mathcal{X})$ classifies the selected example as the desired target, i.e., $f(\boldsymbol{x}^*) = y^*$. A constraint must be imposed to ensure that the number of poisoned samples $|\mathcal{S}_\mathcal{P}| \leq \mathcal{P}\% \times \mathcal{D}_{train}$ does not surpass a specified proportion $\mathcal{P}\%$ of the total training data size $\mathcal{D}_{train}$, where $\mathcal{D}_{train} = \mathcal{X} \cup \mathcal{S}_\mathcal{P}$. In practice, the adversary is typically limited in the amount of data they can poison, often less than 30% of the entire training set [35], [36].

In the experiment, malicious samples exhibiting a high degree of similarity to normal training samples were meticulously selected for poisoning attacks to evade direct elimination during the data cleaning process [21], [37]. These samples were then uniformly treated as normal samples through label-flipping attacks and incorporated into training sessions. Additionally, considering that the poisoned samples are malicious samples very similar to normal data and that the amount of data limits the intensity of the poisoning attack, the poisoning proportion in the experiment was controlled to be between 0% $\sim$ 20%. This is consistent with the poisoning proportion of less than 30% reported in the literature.

B. Capability of Defender

The acquisition of target model knowledge and training data by defenders depends on the threat model. Deep-kNN [33] assumes access to ground-truth labels, and Taheri et al. [34] proposed label-based semi-supervised defense and clustering-based semi-supervised defense. Similar to [34], PRAD operates under the assumption that an adversary has access to trusted training data and utilizes this access to acquire the distribution of normal training data, thereby enabling the forging of poisoned samples that closely resemble legitimate data. The capabilities of defenders may vary under different threat models, as defense may require access to corresponding resources.

IV. Architecture of PRAD

A. Overview of PRAD

An overview of PRAD is shown in Fig. 2. The PRAD comprises a front-end Data Preprocessing Module (DPM) and three primary back-end algorithm modules: Feature Encoding Module (FEM), Feature Analysis Module (FAM), and GMM-based online clustering Anomaly Detection Module (ADM). Specifically, FEM extracts the low-dimensional hidden layer feature representation $\boldsymbol{z}$ and the reconstruction error $\boldsymbol{e}$ from the input sample $\boldsymbol{x}$. FAM then analyzes the differences in features and reconstruction losses between normal and abnormal traffic data. Subsequently, ADM employs the GMM clustering algorithm to cluster the hidden layer features $\boldsymbol{z}$ and assigns different clusters based on the average reconstruction loss $\boldsymbol{\mu}_{e_{ci}}$. Therefore, PRAD captures feature dependencies by combining prior knowledge and clustering algorithms to efficiently detect traffic anomalies in poisoning attack scenarios.

B. Feature Encoding Module

The FEM is used to extract the hidden layer feature representation $\boldsymbol{z}$ from the input data $\boldsymbol{x}$ to alleviate algorithmic complexity. The FEM is constructed using a variant of the VRNN architecture [38], which comprises three primary components: encoder, decoder, and variational lower bound. Specifically, the FEM employs encoder to extract the latent representation $\boldsymbol{z}$ from the original high-dimensional data $\boldsymbol{x}$. Subsequently, decoder reconstructs the original data, providing the reconstruction error $\boldsymbol{e}$ and the reconstruction error residual $\boldsymbol{\varepsilon}$. The variational lower bound is employed to evaluate the convergence of the algorithm. $\boldsymbol{z}$, $\boldsymbol{e}$, and $\boldsymbol{\varepsilon}$ jointly characterize the deviations of anomalous samples from the normal data distribution. ADM trained with these three factors is expected to achieve enhanced generalization capabilities on unseen data.

1) Encoder: Given $\boldsymbol{N}$ multivariate time series data of normal traffic flow, a set of observations is represented by $\boldsymbol{x} = \{\boldsymbol{x}^{(n)}\}_{n=1}^N$. For each flow, $\boldsymbol{x}^{(n)} = \{x_1^n, \ldots, x_t^n, \ldots, x_T^n\} \in \mathbb{R}^T$, where T represents the length of the stream. To infer the current data $\boldsymbol{x}^{(n)}$, we employ FEM for encoding, which also updates the hidden variable $\boldsymbol{h}$ at each time step t. Simultaneously, the prior probability $p(z_t|x_{<t}, z_{<t})$ of z_t is obtained. We input the hidden content of the previous time step $\boldsymbol{h}_{t-1}$ into the prior network to generate the mean $\mu_{pri,t}$ and standard deviation $\sigma_{pri,t}$ of the data distribution. Subsequently, we apply this to a random sampling of the standard normal distribution to obtain subsequent output calculations and hidden state updates. Therefore, we have:

$$\begin{aligned} z_t^n &\sim \mathcal{N}\left(\mu_{pri,t}, \sigma_{pri,t}^2\right), \\ [\mu_{pri,t}, \sigma_{pri,t}] &= \varphi_{pri}\left(\boldsymbol{h}_{t-1}\right), \end{aligned} \tag{2}$$

where $\mu_{pri,t}$ and $\sigma_{pri,t}$ represent the mean and standard deviation of the prior conditional probability, and φ represents the parameters obtained by the encoder. By inputting data x_t^n into the encoder, we obtain our hidden layer feature representation

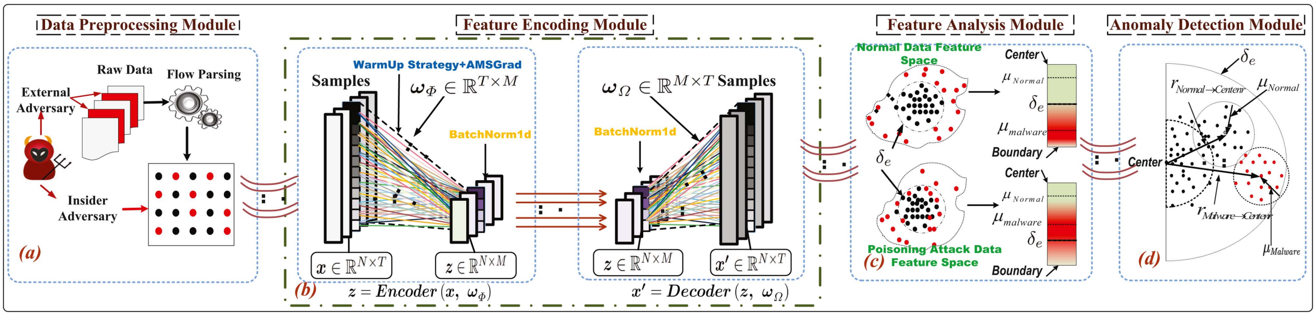

Fig. 2. PRAD consists of four parts, i.e., (a) Data Preprocessing Module (DPM); (b) Feature Encoding Module (FEM); (c) Feature Analysis Module (FAM); and (d) Anomaly Detection Module (ADM).

z_t^n through the encoder:

$$z_t^n | x_t^n \sim \mathcal{N}\left(\mu_{enc}, \sigma_{enc}^2\right),$$
$$\left[\mu_{enc}, \sigma_{enc}^2\right] = \varphi_{enc}\left(\boldsymbol{h}_{t-1}, \varphi_x\left(x_t\right)\right). \tag{3}$$

2) Decoder: The decoder's decoding process aims to reconstruct the input data x_t^n. Its output depends not only on z_t^n but also on the hidden layer variable $\boldsymbol{h}_{t-1}$, we have:

$$x_t^n | z_t^n \sim \mathcal{N}\left(\mu_{dec,t}, \sigma_{dec,t}^2\right),$$
$$\left[\mu_{dec,t}, \sigma_{dec,t}^2\right] = \varphi_{dec}\left(\varphi_{\boldsymbol{z}}\left(z_t^n\right), \boldsymbol{h}_{t-1}\right), \tag{4}$$

where $\varphi_{\boldsymbol{x}}$ and $\varphi_{\boldsymbol{z}}$ are feedforward neural networks used to extract features from $\boldsymbol{x}^{(n)}$ and $\boldsymbol{z}^{(n)}$, respectively. Finally, a GRU neural network is employed to update $\boldsymbol{h}_t$:

$$h_t = \varphi_{\text{GRU}}\left(\boldsymbol{x}_t, \boldsymbol{z}_t, \boldsymbol{h}_{t-1}\right). \tag{5}$$

3) Variational Lower Bound: FEM employs VRNN to enhance the accuracy of prior probability prediction, thereby improving data reconstruction and obtaining the correct hidden layer representation $\boldsymbol{z}^{(n)}$. The Evidence Lower Bound (ELBO) determines its lower bound, so our loss function $\boldsymbol{L}_{loss}$ is the negative of the optimal lower bound:

$$\boldsymbol{L}_{loss} = -\sum_{t=1}^{T} E\left[\log\ p\left(\boldsymbol{x}_t | \boldsymbol{z}_{<t}, \boldsymbol{x}_{<t}\right) - KL\left(\mathcal{N}_{enc,t} || \mathcal{N}_{pri,t}\right)\right], \tag{6}$$

which the first term is to minimize the reconstruction error $\log\ p(\boldsymbol{x}_t | \boldsymbol{z}_{<t}, \boldsymbol{x}_{<t})$, while the second term aims to make the posterior distribution $\mathcal{N}_{enc,t}$ closely approximate the prior distribution $\mathcal{N}_{pri,t}$.

Leveraging FEM, the high-dimensional original data $\boldsymbol{x}$ is reconstructed into $\boldsymbol{x'}$ and the hidden layer feature representation $\boldsymbol{z} = \{f_{\mathcal{T}s}(\boldsymbol{x}) | \boldsymbol{z} \in \mathbb{R}^M\}$ $(M \ll N)$. Additionally, another key element is obtained, which is the reconstruction error e_i:

$$e_i = \|\boldsymbol{x}_i - \boldsymbol{x'}_i\|, \tag{7}$$

where $\|\cdot\|$ is the L2-norm, and e_i also serves as an indicator of anomalous samples. Thus, the set of all reconstruction errors is obtained: $\boldsymbol{e} = \{e_1, \cdots e_N\} \in \mathbb{R}^N$. We select the maximum reconstruction error δ_e as the threshold to calculate our final required reconstruction residual ε_i as follows:

$$\varepsilon_i = e_i - \delta_e, \tag{8}$$

where ε_i is represents the difference between the reconstruction error of the data set and the maximum reconstruction error in the training set. Essentially, it indicates the direction of the input data. To sum up, $\boldsymbol{z}$, $\boldsymbol{e}$, and $\boldsymbol{\varepsilon}$ characterize the input samples. Notably, these three elements uniquely represent the input samples. With the FEM module, typical anomalies with large reconstruction errors can deviate significantly from normal samples.

C. Feature Analysis Module

The purpose of FAM is to analyze the impact of poisoning attacks on the zero-positive training set. We analyze the impact of the poisoned model on the unlabeled data set $\mathcal{D}_{test}$ from two perspectives: (1) the changes in the feature space caused by the poisoned model, and (2) the changes in the distribution of the reconstruction error loss. This analysis also provides a foundation for subsequent ADM anomaly identification by providing a priori information. Consider a pure training set $\mathcal{D}_{train}$ and a poisoned training set $\mathcal{D}'_{train}$. The training processes of the pure training set and the poisoned training set can be simplified to $\mathcal{T}_s(\mathcal{D}_{train})$ and $\mathcal{T}_s(\mathcal{D}'_{train})$, respectively. The elements of the unlabeled data set $\mathcal{D}_{test}$ obtained by different FAM encoding processes f are $f(\boldsymbol{z}, \boldsymbol{e^T}, \boldsymbol{\varepsilon^T})$ and $f(\tilde{\boldsymbol{z}}, \tilde{\boldsymbol{e}}^{\boldsymbol{T}}, \tilde{\boldsymbol{\varepsilon}}^{\boldsymbol{T}})$, respectively. To quantify the impact of poisoning attacks on normal and abnormal data in unlabeled sample sets, we propose two metrics, i.e., feature-based inter-class distance similarity evaluation, and intra-class reconstruction error distribution-based shift evaluation.

1) Feature-Based Inter-Class Distance Similarity Evaluation: The RL-based method currently employs a classification-based anomaly detection approach to tackle the dimensionality challenge posed by the high-dimensional nature of traffic data. However, the inter-class distance of the representation data in the unlabeled data set obtained through FEM decreases due to the influence of poisoning attacks on the training set. To quantify the changes in the feature space induced by model poisoning, we employ the ratio of the inter-class distance after poisoning to the inter-class distance obtained through normal encoding as a metric for measuring similarity changes. Consider $\boldsymbol{z} = \{\boldsymbol{z_n} \in \mathbb{R}^{n*M}, \boldsymbol{z_m} \in \mathbb{R}^{m*M}\}, (n+m = N_T)$ as the unlabeled data set representation data obtained by pure encoding, where $\boldsymbol{z_n}$ represents the hidden layer representation of normal samples and $\boldsymbol{z_m}$ represents the hidden layer representation of abnormal samples, and M is the dimensionality of hidden layer features. Similarly, $\tilde{\boldsymbol{z}} = \{\tilde{\boldsymbol{z}}_{\boldsymbol{n}} \in \mathbb{R}^{n*M}, \tilde{\boldsymbol{z}}_{\boldsymbol{m}} \in \mathbb{R}^{m*M}\}$ is the unlabeled data set representation data obtained by poisoned encoding. $\{\eta_1, \cdots \eta_n, \eta_{n+1}, \cdots \eta_{n+m}\}$ positive numbers, think of these as weights, or better yet, as "multiplication" of axes. Let $\mathcal{C}_{\boldsymbol{n}}(y_n)$ and $\mathcal{C}_{\boldsymbol{m}}(y_m)$ denote the weighted sum of the distances between the positive class center y_n and the positive data points $\boldsymbol{z_n}$, and the weighted sum of the distances between the abnormal class center y_m and the abnormal data points $\boldsymbol{z_m}$, respectively:

$$\mathcal{C}_{\boldsymbol{n}}(y_n) = \sum_{i=1}^{n} \eta_i d_i(y_n),\ \mathcal{C}_{\boldsymbol{m}}(y_m) = \sum_{i=n+1}^{n+m} \eta_i d_i(y_m), \tag{9}$$

where $d_i(y) = \|y - z_i\|$ represents the Euclidean distance from the class center y to z_i. The objective is to identify a point y (or a set of points) that minimizes the specified "cost function" $\mathcal{C}(y)$:

$$G_{\boldsymbol{n}}(\boldsymbol{z_n}, \eta_n) = \arg\min\left(\mathcal{C}_{\boldsymbol{n}}(y_n), y \in \mathbb{R}^M\right), \tag{10}$$

$$G_{\boldsymbol{m}}(\boldsymbol{z_m}, \eta_m) = \arg\min\left(\mathcal{C}_{\boldsymbol{m}}(y_m), y \in \mathbb{R}^M\right), \tag{11}$$

where G is referred to as the weighted geometric median [39], which employ the improved Weiszfeld algorithm [40] to solve this problem. Subsequently, the impact of the poisoning attack on the unlabeled data set $\mathcal{D}_{test}$ can be assessed by calculating the Euclidean distance $E(G_{\boldsymbol{n}}, G_{\boldsymbol{m}})$ between the normal data distribution $G_{\boldsymbol{n}}$ and the distribution of poisoned data $G_{\boldsymbol{m}}$, which is obtained using the improved Weiszfeld algorithm. This allows us to compare the inter-class distance of the pure encoded data $E(G_{\boldsymbol{n}}, G_{\boldsymbol{m}})$ with the inter-class distance of the poisoned encoded data $\tilde{E}(G_{\boldsymbol{n}}, G_{\boldsymbol{m}})$, thereby quantifying the impact of poisoning attacks. Furthermore, to gain a clearer understanding of the change in inter-class distance after surface poisoning, we opt to measure similarity based on distance.

$$\boldsymbol{Sim} = \frac{\widetilde{\mathbb{E}}(G_n, G_m)}{\mathbb{E}(G_n, G_m)}, \tag{12}$$

where "$\boldsymbol{Sim}$" represents the feature-based inter-class distance "similarity" evaluation index. A smaller value indicates a smaller class distance, making the classification task of normal and abnormal data more challenging. Section V-D1 presents the experimental analysis of feature space similarity evaluation for unlabeled datasets under poisoning attacks.

2) Intra-Class Reconstruction Error Distribution-Based Shift Evaluation: Reconstruction errors are commonly employed as indicators for anomaly detection. However, anomaly detection methods based on reconstruction errors become ineffective when the training data is compromised by poisoning attacks. In this work, we propose a strategy based on reconstruction error distribution shift evaluation to quantify the impact of poisoning attacks on unlabeled datasets $\mathcal{D}_{test}$. Specifically, we measure the impact of training data poisoning by comparing the changes in the reconstruction errors distribution across different data classes in the unlabeled dataset $\mathcal{D}_{test}$. Let $\boldsymbol{e} = \{\boldsymbol{e}_n, \boldsymbol{e}_m\}$ denote the unlabeled dataset $\mathcal{D}_{test}$ reconstruction errors obtained by pure encoding, where n represents the number of normal samples in the unlabeled data set and m represents the number of abnormal samples. Similarly, the unlabeled dataset $\mathcal{D}_{test}$ reconstruction errors obtained by poisoned encoding can be obtained: $\tilde{\boldsymbol{e}} = \{\tilde{\boldsymbol{e}}_n, \tilde{\boldsymbol{e}}_m\}$. The distribution of intra-class reconstruction errors can then be measured by calculating the mean μ and variance σ^2:

$$\mu_{\boldsymbol{n}} = \sum_{i=1}^{n} e_i, \quad \sigma_{\boldsymbol{n}}^2 = \frac{1}{n}\sum_{i=1}^{n}(e_i - \mu_{\boldsymbol{n}})^2, \tag{13}$$

$$\mu_{\boldsymbol{m}} = \sum_{i=1}^{m} e_i, \quad \sigma_{\boldsymbol{m}}^2 = \frac{1}{m}\sum_{i=1}^{m}(e_i - \mu_{\boldsymbol{m}})^2. \tag{14}$$

By comparing the reconstruction error distributions of different data types in the unlabeled dataset $\mathcal{D}_{test}$ obtained through pure encoding and poisoning encoding, we can quantify the impact of poisoning attacks on the reconstruction error distribution of various data types in the unlabeled dataset $\mathcal{D}_{test}$. It is noteworthy that under the influence of poisoning attacks, as the attack intensity increases, the average reconstruction loss distribution of abnormal data in the unlabeled dataset becomes increasingly similar to the average reconstruction loss distribution of normal data, making the anomaly detection classification task progressively more challenging. Our subsequent classification hypothesis posits that the average reconstruction error distribution of abnormal data is higher than that of normal data. In the entire unlabeled dataset, the number of abnormal data samples is smaller than that of normal samples. Therefore, the bias based on the intra-class reconstruction error distribution-based shift evaluation also provides valuable prior information for subsequent anomaly detection algorithms. The experimental results are analyzed in Section V-D2, which corroborate our hypothesis.

D. Abnormal Detection Module

Leveraging the a priori information provided by FAM and the features encoded by FEM, we propose an ADM based on GMM online clustering. Specifically, the ADM employs an online GMM clustering algorithm to cluster the encoded features $\boldsymbol{z}$. Subsequently, it assigns different clusters based on the average reconstruction loss $\mu_{e_{ci}}$, thereby achieving anomaly detection. The detailed process of the ADM module is illustrated in Fig. 3.

Consider a set of J mixed multivariate Gaussian distributions, indexed by the parameter set $\Theta = \{\alpha_j, \vartheta_j\}$, where $\vartheta_j = \{\mu_j, \Sigma_j\}$ denote the parameters of the j-th Gaussian distribution, where the mean vector is denoted by μ_j, Σ_j is the covariance matrix, and $\alpha_j \in [0, 1]$ is the mixing probability. Suppose that the encoded feature sets $\boldsymbol{z} \in \mathbb{R}^{N_T*M}$ obtained by applying FEM to the original data $\boldsymbol{x} \in \mathbb{R}^{N_T*T}$ are i.i.d according to the mixed probability density function. We estimate the parameters Θ using the maximum likelihood estimation procedure,

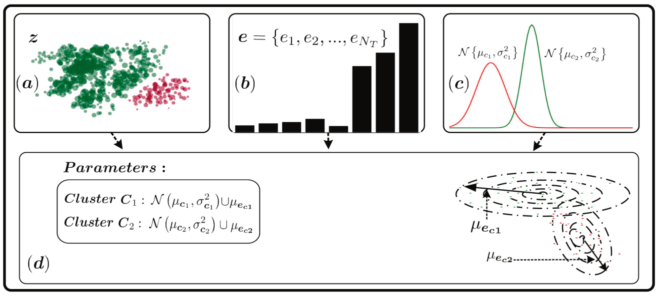

Fig. 3. The ADM consists of (a) encoded feature set; (b) reconstruction error set; and (c) online GMM clustering algorithm. The last (d) is the cluster result obtained by online GMM clustering.

as shown in (15), where the log-likelihood is given by (16):

$$L(\Theta|\boldsymbol{z}) = \prod_{i \in N_T} p(\boldsymbol{z_i}|\vartheta_j) = \prod \left(\sum_{j \in J} \alpha_j p_j (\boldsymbol{z_i}|\mu_j, \Sigma_j) \right), \tag{15}$$

$$\log(\Theta|\boldsymbol{z}) = \sum_{i \in N_T} \log \left(\sum_{j \in J} \alpha_j p_j (\boldsymbol{z_i}|\mu_j, \Sigma_j) \right), \tag{16}$$

where Σ_j is a symmetric positive definite matrix, $\boldsymbol{z_i}$ represents the observable elements of the i-th sample, $p_j(\boldsymbol{z_i}|\mu_j, \Sigma_j)$ represents the probability density of the j-th sample value corresponding to the j-th Gaussian component, and its mathematical expression is as follows:

$$P(\boldsymbol{z_i}|\mu_j, \Sigma_j) = \frac{1}{(2\pi)^{N_T/2} |\Sigma_j|^{1/2}} e^{-1/2(\boldsymbol{z_i}-\mu_j)^T \Sigma_j^{-1}(\boldsymbol{z_i}-\mu_j)}. \tag{17}$$

Given the structural complexity of (15), the optimal Θ cannot be obtained by setting the derivative to zero. The Expectation Maximization (EM) procedure [41] is a powerful method for maximizing the log-likelihood function (16) and finding the optimal parameters. The latter iteratively updates the parameters of each Gaussian distribution. Therefore, the parameter set of GMM consists of $\{\alpha_j, \mu_j, \Sigma_j\}$, where $1 \leq j \leq J$. And the parameters are estimated under the maximum likelihood setting. Optimization is usually performed using the EM algorithm, which is divided into two steps, i.e., E-step and M-step:

1) *E-step:* The posterior probability γ_{ij} is calculated by the equation, according to the given value of $\{\alpha_j, \mu_j, \Sigma_j\}$:

$$\gamma_{ij} = \frac{\alpha_j p(\boldsymbol{z_i}|\mu_j, \Sigma_j)}{\sum_{j=1}^{J} \alpha_j p(\boldsymbol{z_i}|\mu_j, \Sigma_j)}. \tag{18}$$

2) *M-step:* A new set of $\{\alpha_j, \mu_j, \Sigma_j\}$ can be obtained by using the aforementioned γ_{ij}:

$$\alpha_j = \frac{\Phi_{ij}}{N_T}, \quad \mu_j = \frac{1}{\Phi_{ij}} \sum_{j=1}^{N_T} \gamma_{ij} \boldsymbol{z_i}, \tag{19}$$

$$\Sigma_j = \frac{1}{\Phi_{ij}} \sum_{j=1}^{N_T} \gamma_{ij} (\boldsymbol{z_i} - \mu_j)(\boldsymbol{z_i} - \mu_j)^T, \tag{20}$$

where $\Phi_{ij} = \sum_{j=1}^{N_T} \gamma_{ij}$. E-step and M-step are calculated iteratively until the condition converges according to the likelihood function of the formula (16).

Employing the online GMM clustering algorithm, J ($J = 2$) clusters are obtained. Subsequently, the a priori information provided by the FAM is leveraged to identify the corresponding reconstruction loss for the feature representation in each cluster. The average reconstruction loss $\mu_{e_{cj}}$ of the two clusters is then calculated, respectively:

$$\mu_{e_{c1}} = \sum_{i=1}^{n_1} e_{ci}, \quad \mu_{e_{c2}} = \sum_{i=1}^{n_2} e_{ci}, \tag{21}$$

where $\mu_{e_{c1}}$ and $\mu_{e_{c2}}$ represent the average reconstruction loss of the two clusters, respectively, and n_1 and n_2 represent the amount of data contained in each cluster, respectively. The average loss is compared with the a priori information provided by FAM. If $\mu_{e_{c1}} > \mu_{e_{c2}}$, then the cluster corresponding to $\mu_{e_{c2}}$ is considered a normal sample, and the cluster corresponding to $\mu_{e_{c1}}$ is an abnormal sample. Conversely, if $\mu_{e_{c1}} < \mu_{e_{c2}}$, then the cluster corresponding to $\mu_{e_{c1}}$ is considered a normal sample, and the cluster corresponding to $\mu_{e_{c2}}$ is an abnormal sample. The pseudocode for the online clustering ADM is shown in **Algorithm 1**.

E. Training Procedure

A two-stage training strategy is employed to construct PRAD. In the first stage, pre-training is performed using only the reconstruction loss to train the feature encoding network. During this stage, all unlabeled training samples are utilized to pre-train the FEM with the objective of obtaining the entire network for subsequent stages. The fundamental encoding strategy involves leveraging Amsgrad stochastic gradient descent algorithm and the warm-up strategy to train the network, thereby mitigating the issue of shrinking inter-class distance caused by poisoning attacks. In the second stage, FAM is employed to acquire prior knowledge of the data for the subsequent ADM module by utilizing the extracted features and their corresponding reconstruction errors. The limited effectiveness of reconstruction error as a sole classification criterion is addressed by utilizing a GMM to cluster the extracted features. This approach involves assigning cluster labels based on the reconstruction error values, thereby enhancing the classification process.

V. Experiments

A. Dataset Description

In this part of the experiments, the performance of PRAD is assessed on two publicly available encrypted traffic anomaly detection datasets, i.e., CTU13 [42] and CIRA-CIC-DoHBrw-2020 (DoH2020) [43]. Specifically, the CTU13 dataset consists of botnet traffic combined with normal and background traffic, encompassing 13 types of botnet traffic. The DoH2020 dataset

Algorithm 1: Online Clustering ADM.

Input: Encoded feature set
$\boldsymbol{z} = \{\boldsymbol{z}_1, \boldsymbol{z}_2, \ldots, \boldsymbol{z}_{\boldsymbol{N_T}}\} \in \mathbb{R}^{N_T \times M}$;
The number of clusters J; Reconstruction errors set of $\boldsymbol{z}$:
$\boldsymbol{e} = \{e_1, e_2, \ldots, e_{N_T}\} \in \mathbb{R}^{N_T}$.
Output: $\{\alpha_1, \ldots, \alpha_J\}$; $\{(\mu_1, \Sigma_1), \ldots, (\mu_J, \Sigma_J)\}$; AD.
1. Initialize the parameters of GMM according to $\boldsymbol{z}$.
2. **Loop Body**:
   E-step: Calculate the posterior probability γ_{ij} by (18)
   M-step: Re-estimate the parameters using data samples weighted by the posterior probabilities using formulas (19) and (20).
3. **Until** the maximum iteration number $r_{\max}$ or the termination condition is reached.
4. Output 1: The parameter sets of each GMM component $\{(\mu_1, \Sigma_1), \ldots, (\mu_J, \Sigma_J)\}$ and $\{\alpha_1, \ldots, \alpha_J\}$.
5. Count the number of samples in each cluster, and find the corresponding reconstruction error in $\boldsymbol{e}$.
6. Calculate the average reconstruction error of samples in each cluster according to formula (21), recorded as $\mu_{e_{c1}}$ and $\mu_{e_{c1}}$, respectively.
7. **While** $\mu_{e_{c1}} \neq \mu_{e_{c2}}$ **do**
   If $\mu_{e_{c1}} > \mu_{e_{c2}}$ **then**
      $c1$ is an abnormal sample cluster;
      $c2$ is a normal sample cluster;
   else
      $c1$ is a normal sample cluster;
      $c2$ is an abnormal sample cluster;
   end while
8. Output 2: AD

TABLE I
DATASETS DETAILS

Datasets	Train Sets			Unlabeled set		
	Normal	Poision ($\mathcal{S}_{\mathcal{P}}$)	$\mathcal{P}\%$	Normal	Abnormal	η
DOH2020	250000	0	0%	309037	71101	18.7%
	247500	2500	1%	309037	71101	18.7%
	245000	5000	2%	309037	71101	18.7%
	237500	12500	5%	309037	71101	18.7%
	145602	16178	10%	309037	71101	18.7%
	91675	16178	15%	309037	71101	18.7%
	64712	16178	20%	309037	71101	18.7%
CTU13	48580	0	0%	48580	12496	20.5%
	48094	485	1%	48580	12496	20.5%
	47608	971	2%	48580	12496	20.5%
	46151	2429	5%	48580	12496	20.5%
	43722	4858	10%	48580	12496	20.5%
	41293	7287	15%	48580	12496	20.5%
	38864	9716	20%	48580	12496	20.5%

facilitates the analysis, testing, and evaluation of DoH traffic in covert channels and tunnels, and can be utilized to assess encrypted traffic anomaly detection algorithms. In our experiments, we carefully selected hundreds of thousands of data points from the two datasets for further experimentation. Prior to employing these datasets for anomaly detector evaluation, standard preprocessing procedures were applied to each dataset. Firstly, the initial 500 bytes of each dataset's individual data stream payload were extracted. Subsequently, excessively long data streams were truncated, and excessively short data streams were padded, followed by data normalization. Table I presents the details of the datasets, each dataset consists of a training set and a unlabeled data set. The training set includes normal samples and carefully selected poisoned samples, while the unlabeled data set comprises a specific proportion of positive samples and negative samples. Here, $\mathcal{P}\%$ denotes the ratio of poisoned data in the training set, and η represents the proportion of abnormal samples in the unlabeled data set.

B. Performance Metrics

To gauge prediction performance, the AUCROC metric is employed, which is particularly susceptible to poisoning attacks. Additionally, AUCPR is utilized to account for sample imbalance in measuring prediction performance. AUCROC represents the area under the receiver operating characteristic curve, while AUCPR denotes the area under the precision-recall curve. AUCROC characterizes the relationship between the true positive rate and false positive rate, whereas AUCPR focuses on the association between precision and recall. While AUCROC evaluates prediction performance for both normal and anomaly categories, AUCPR emphasizes anomaly-centric performance. It's worth noting that both AUCROC and AUCPR range from 0 to 1, with higher values indicating superior performance.

C. Competitive Methods and Parameter Settings for PRAD

1) Competitive Methods: PRAD is compared with nine methods: ECOD [6], COPOD [7], KDE [44], OCSVM [5], LOF [4], iForest [8], LSCP [9], INNE [45], and SUOD [10]. Among these methods, KDE, LOF, and IForest do not employ poisoning attack suppression strategies, while the other six methods all incorporate such strategies. All comparison algorithms are implemented using Python's PyOD library, and the default parameters are utilized for all methods. Notably, OCSVM offers two poisoning suppression strategies ($\mu = 0.1$ and $u = 0.5$). PRAD is implemented using PyTorch, and experiments are conducted using the PyTorch 2.0.0 framework via PyCharm 2022.2.5 (Community Edition) on computers equipped with a GeForce RTX 3090 GPU and an Intel (R) Xeon (R) Silver 4216 CPU.

2) Parameter Settings: By default, the input encoder and prior encoding in PRAD's FEM consist of two linear encodings and a layer of Batchnorm1D regularization. The decoder is similar, with a GRU neural network as the middle hidden layer. The activation function for the middle layer is ReLU, while the final output layer employs the Sigmoid function as its activation function. The input data dimension is 500, and the dimensionality of the encoded data is 128. It's worth noting that varying the number of hidden layers in FEM ($N = 4$ by default) can impact the performance of PRAD. The proposed PRAD is trained for a total of 175 epochs with a batch size of 128, warm-up is 5 epochs. Validation results obtain at the end of each training phase serve to assess the PRAD's generalization capability and aid in deciding whether to retain the trained FEM. The clustering parameter J is set to 2.

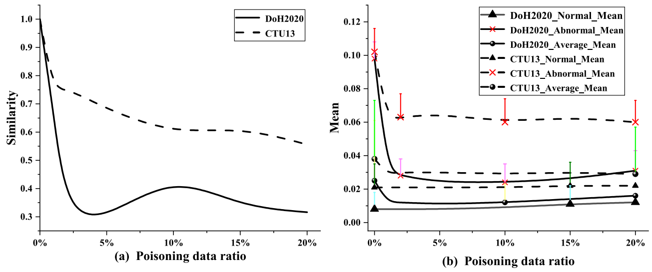

Fig. 4. Feature analysis result: (a) feature-based inter-class distance similarity evaluation for poisoning impact estimation; (b) intra-class reconstruction error distribution-based shift evaluation for poisoning attack impact estimation.

D. Effect of Feature Analysis

1) Feature-Based Inter-Class Distance Similarity Evaluation: In this part, we employ feature-based inter-class distance similarity to assess the impact of poisoned FEM on unlabeled data sets when the training set data is subject to poisoning attacks. Specifically, the concept of similarity here quantifies the impact of poisoning attacks on the distance between the center of normal samples and the centers of abnormal samples. It is important to note that lower similarity values indicate a reduced distance between data points of different classes, posing significant challenges for subsequent anomaly detection tasks. Fig. 4(a) illustrates that as the intensity of poisoning attacks in the training set increases, the impact of poisoning attacks intensifies, leading to a continuous decrease in the center distance between positive samples and abnormal samples in the unlabeled data set. The DOH2020 experiences the most significant impact, with the distance between the centers of positive samples and abnormal samples reduced to only 31% of the original class distance. Similarly, in the CTU13, the distance between normal samples and abnormal samples is reduced to approximately 56% of the original class distance. The feature-based inter-class distance similarity evaluation demonstrates that the impact of poisoning attacks on the training set leads to a continuous reduction in the class distance of positive anomaly data in the unlabeled dataset, which poses significant challenges for subsequent anomaly detection tasks.

2) Intra-Class Reconstruction Error Distribution-Based Shift Evaluation: In this part, we employ shift evaluation based on intra-class reconstruction error distribution to quantify the impact of the poisoned FEM module on the reconstruction error distribution of positive anomaly samples in the unlabeled data set when the training set is compromised by poisoning attacks. Fig. 4(b) presents error bar plots depicting the reconstruction errors for different components of the unlabeled samples in the DOH2020 and CTU13. As evident from the Fig. 4(b), when FEM is not affected by poisoning, the reconstruction errors for normal samples and abnormal samples in the DOH2020 unlabeled dataset are 0.01 and 0.1, respectively, with an overall unlabeled dataset error of 0.025. Similarly, in the absence of poisoning attacks on FEM, the reconstruction errors in CTU13 are 0.02, 0.1, and 0.04 for normal samples, abnormal samples, and the entire unlabeled dataset, respectively. When the training set is subjected to poisoning attacks, each component of the unlabeled dataset encoded by the poisoned FEM undergoes changes. Specifically, the average reconstruction error distribution of abnormal samples exhibits a continuous decrease, while the reconstruction error distribution of positive samples remains largely unchanged. More specifically, in the unlabeled sample sets of DOH2020 and CTU13, the average reconstruction error distribution of normal samples remains stable at approximately 0.01 and 0.02, respectively. In contrast, the average reconstruction error distribution of abnormal samples gradually decreases and eventually stabilizes around 0.025 and 0.06, respectively. Concurrently, the mean distribution of the entire unlabeled sample set also exhibits a gradual decrease, ultimately settling around 0.015 and 0.03, respectively. It is noteworthy that the mean reconstruction error of abnormal samples consistently exceeds that of normal samples in both datasets, offering valuable prior information for the proposed ADM.

E. Experiment Results With Competitive Methods

In this part, we compare the proposed PRAD with other competing methods. The performance of different algorithms on data without FEM is evaluated, particularly under varying proportions of poisoning attacks. Subsequently, the performance of feature-encoded data using alternative comparison algorithms is assessed under different degrees of poisoning attacks. PRAD is adopted as a baseline for anomaly detection on poisoned data, addressing the semi-supervised anomaly detection problem trained on the poisoned data in the training set. Comparing the results in Table II, it is evident that traditional anomaly detection algorithms without RL exhibit satisfactory performance on two metrics when faced with data poisoning attacks. However, due to the high dimensionality of data attributes, the training cost of these algorithms becomes prohibitively high. Conversely, when utilizing RL-based solutions, traditional anomaly detection algorithms struggle in the presence of data poisoning attacks.

As the intensity of poisoning attacks escalates, the inter-class distance between normal and abnormal samples progressively diminishes, leading to the failure of traditional anomaly detection algorithms, especially on the CTU13. The average highest AUCROC and AUCPR scores across all compared algorithms on the DoH2020 and CTU13 are 0.821 and 0.533, respectively. Undoubtedly, this poses a significant challenge for the anomaly detection task. In comparison to previous anomaly detection algorithms, the proposed PRAD demonstrates superior robustness against poisoning attacks. Firstly, PRAD maintains excellent performance levels even without data poisoning attacks. Moreover, as the intensity of data poisoning attacks increases, PRAD exhibits remarkable resilience in the face of decreasing inter-class distance. With the increasing proportion of poisoned data, the anomaly detection indicators AUCROC and AUCPR scores stabilize above 0.980 and 0.986, respectively. Importantly, the PRAD emerges as the frontrunner in the presence of various levels of data attack intensity. Additionally, the average AUCROC and AUCPR scores across the two validation datasets reach an astonishing 0.995 and 0.996, respectively. Thus, PRAD achieves the highest average performance among all competing

TABLE II
AUCROC AND AUCPR OF PRAD AND COMPARISON ALGORITHMS UNDER POISONING INTENSITY OF 0% ∼ 20%

Representation Methods	Anomaly Detection	AUC_ROC														Average_AUC_ROC
		DoH2020_Poision_Rate							CTU13_Poision_Rate							/
		0%	1%	2%	5%	10%	15%	20%	0%	1%	2%	5%	10%	15%	20%	/
Data Stand Only	ECOD	0.998	0.998	0.998	0.998	0.996	0.993	0.990	0.996	0.996	0.994	0.973	0.898	0.821	0.728	0.956
	COPOD	0.999	0.999	0.999	0.999	0.998	0.998	0.998	0.847	0.829	0.815	0.775	0.740	0.685	0.602	0.877
	KDE	1.0	0.999	0.999	0.999	0.999	0.999	0.998	1.0	0.734	0.720	0.712	0.701	0.661	0.615	0.867
	OCSVM(μ=0.1)	1.0	0.999	0.999	0.998	0.999	0.985	0.983	1.0	0.999	0.998	0.987	0.984	0.982	0.980	0.992
	OCSVM(μ=0.5)	1.0	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.987	0.860	0.841	0.977
	LOF	0.999	0.321	0.324	0.329	0.291	0.261	0.249	0.998	0.701	0.709	0.714	0.713	0.714	0.712	0.574
	INNE	0.925	0.845	0.844	0.836	0.831	0.823	0.804	0.999	0.998	0.993	0.979	0.952	0.924	0.887	0.903
	IForest	0.999	0.999	0.999	0.999	0.999	0.997	0.996	0.995	0.999	0.999	0.968	0.723	0.587	0.534	0.914
	LSCP	0.997	0.953	0.907	0.899	0.779	0.870	0.857	0.986	0.956	0.954	0.944	0.864	0.797	0.787	0.896
	SUOD	0.998	0.998	0.995	0.993	0.960	0.924	0.887	0.995	0.997	0.995	0.933	0.833	0.806	0.764	0.934
Feature Encoding (VRNN)	ECOD	0.973	0.839	0.788	0.812	0.852	0.910	0.970	0.852	0.611	0.571	0.575	0.419	0.409	0.426	0.715
	COPOD	0.985	0.910	0.886	0.910	0.913	0.928	0.992	0.870	0.628	0.555	0.567	0.444	0.358	0.384	0.738
	KDE	0.753	0.688	0.658	0.719	0.678	0.684	0.737	0.818	0.665	0.632	0.643	0.595	0.603	0.610	0.677
	OCSVM(μ=0.1)	0.999	0.922	0.905	0.908	0.896	0.871	0.971	0.987	0.746	0.723	0.753	0.603	0.645	0.567	0.821
	OCSVM(μ=0.5)	0.992	0.928	0.904	0.909	0.912	0.928	0.984	0.973	0.692	0.680	0.669	0.571	0.536	0.557	0.803
	LOF	0.991	0.630	0.595	0.472	0.499	0.628	0.956	0.905	0.623	0.661	0.632	0.624	0.571	0.590	0.670
	INNE	0.994	0.926	0.905	0.908	0.903	0.924	0.969	0.983	0.773	0.746	0.773	0.566	0.450	0.544	0.812
	IForest	0.991	0.901	0.839	0.895	0.911	0.942	0.991	0.924	0.690	0.650	0.665	0.484	0.435	0.471	0.771
	LSCP	0.943	0.869	0.793	0.852	0.869	0.909	0.978	0.863	0.646	0.596	0.596	0.476	0.427	0.477	0.735
	SUOD	0.996	0.909	0.886	0.887	0.910	0.920	0.989	0.911	0.652	0.630	0.606	0.471	0.428	0.446	0.760
	Average	0.962	0.852	0.816	0.827	0.834	0.864	0.956	0.909	0.673	0.644	0.648	0.525	0.486	0.507	/
	Proposed PRAD (Ours)	0.999	0.993	0.988	0.999	0.980	0.980	0.998	1.0	0.998	0.999	0.999	0.999	0.999	0.999	0.995

Representation Methods	Anomaly Detection	AUC_PR														Average_AUC_PR
		DoH2020_Poision_Rate							CTU13_Poision_Rate							/
		0%	1%	2%	5%	10%	15%	20%	0%	1%	2%	5%	10%	15%	20%	/
Data Stand Only	ECOD	0.973	0.972	0.972	0.970	0.999	0.938	0.921	0.953	0.947	0.937	0.841	0.633	0.490	0.378	0.852
	COPOD	0.982	0.982	0.982	0.981	0.979	0.977	0.976	0.411	0.386	0.368	0.325	0.296	0.261	0.225	0.6522
	KDE	1.0	0.999	0.994	0.988	0.985	0.984	0.981	1.0	0.650	0.646	0.643	0.640	0.630	0.621	0.8402
	OCSVM(μ=0.1)	1.0	0.999	0.999	0.981	0.999	0.888	0.871	1.0	0.999	0.990	0.940	0.929	0.923	0.916	0.960
	OCSVM(μ=0.5)	1.0	0.999	0.999	0.999	0.998	0.993	0.984	1.0	0.999	0.999	0.997	0.937	0.701	0.684	0.949
	LOF	0.999	0.138	0.138	0.139	0.133	0.129	0.127	0.966	0.613	0.616	0.617	0.617	0.626	0.624	0.463
	INNE	0.606	0.470	0.423	0.406	0.380	0.360	0.340	0.994	0.976	0.928	0.861	0.790	0.742	0.700	0.641
	IForest	0.998	0.999	0.999	0.999	0.998	0.986	0.983	0.958	0.994	0.995	0.861	0.626	0.598	0.597	0.899
	LSCP	0.986	0.760	0.529	0.655	0.351	0.449	0.407	0.918	0.799	0.813	0.764	0.590	0.606	0.592	0.659
	SUOD	0.981	0.981	0.955	0.943	0.709	0.559	0.453	0.944	0.957	0.945	0.764	0.605	0.630	0.600	0.788
Feature Encoding (VRNN)	ECOD	0.778	0.395	0.312	0.394	0.459	0.556	0.832	0.447	0.227	0.214	0.212	0.167	0.161	0.170	0.380
	COPOD	0.902	0.506	0.461	0.508	0.510	0.551	0.933	0.605	0.233	0.208	0.209	0.170	0.151	0.160	0.436
	KDE	0.649	0.310	0.284	0.454	0.457	0.469	0.635	0.430	0.321	0.305	0.310	0.262	0.260	0.266	0.387
	OCSVM(μ=0.1)	0.992	0.566	0.494	0.489	0.476	0.468	0.769	0.936	0.421	0.371	0.372	0.283	0.259	0.268	0.512
	OCSVM(μ=0.5)	0.994	0.569	0.493	0.500	0.507	0.552	0.855	0.881	0.298	0.306	0.278	0.225	0.198	0.210	0.490
	LOF	0.885	0.234	0.215	0.163	0.258	0.236	0.824	0.538	0.288	0.335	0.282	0.282	0.227	0.240	0.358
	INNE	0.963	0.553	0.496	0.498	0.488	0.541	0.759	0.903	0.550	0.524	0.450	0.318	0.172	0.247	0.533
	IForest	0.957	0.234	0.396	0.494	0.517	0.607	0.924	0.748	0.286	0.264	0.282	0.186	0.169	0.192	0.447
	LSCP	0.620	0.424	0.398	0.404	0.428	0.506	0.831	0.491	0.242	0.224	0.220	0.182	0.165	0.184	0.380
	SUOD	0.977	0.504	0.458	0.456	0.504	0.527	0.906	0.661	0.246	0.245	0.227	0.183	0.166	0.182	0.446
	Average	0.872	0.430	0.401	0.436	0.460	0.501	0.827	0.664	0.311	0.300	0.284	0.226	0.193	0.188	/
	Proposed PRAD (Ours)	0.994	0.986	0.990	0.999	0.995	0.996	0.998	1.0	0.996	0.997	0.999	0.999	0.999	0.999	0.996

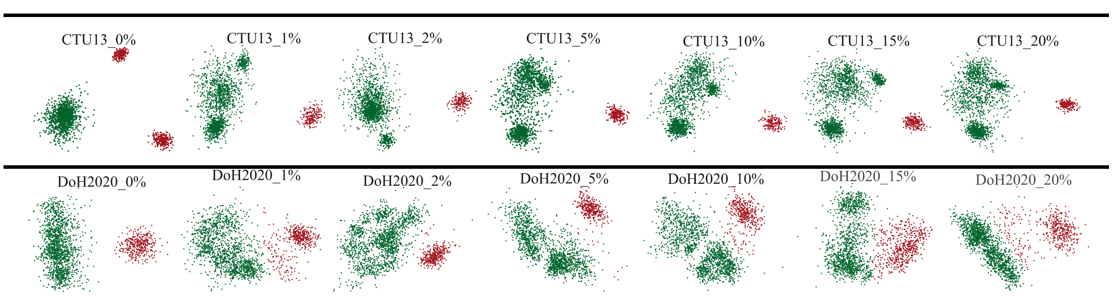

Fig. 5. Clustering results of CTU13 and DoH2020 under different poisoning attack intensities.

approaches. Fig. 5 illustrates the clustering effect of PRAD on the CTU13 and DoH2020. In the absence of data poisoning attacks, the inter-class distance between positive and negative data consistently exhibits a clear separation. However, as the intensity of data poisoning attacks gradually intensifies, the distance between classes gradually diminishes, causing abnormal data points to move closer to normal data points.

F. Ablative Analysis

1) The Effects of Different Encoding Strategies: We use different encoding strategies to demonstrate the rationality of selecting VRNN as FEM within the PRAD framework. For comparison, we assess the performance of AE and VAE using the same experimental setup as the VRNN FEM. Table III presents the experimental results of the nine comparison algorithms. When AE and VAE are employed as FEM and are not subjected to data poisoning attacks, their subsequent anomaly detection performance surpasses that of traditional anomaly detection algorithms, as evident from Table III. Furthermore, their simple architecture and low computational complexity make them suitable options, especially when handling low-intensity data poisoning attacks. This flexibility enables the FEM to be seamlessly integrated as a plug-and-play component. However,

TABLE III
COMPARISON OF PERFORMANCE BETWEEN DIFFERENT COMPARISON ALGORITHMS

Datasets (Representation Method)	Anomaly Detection	AUC_ROC							Average AUC_ROC	AUC_PR							Average AUC_PR
		Poisoning_Rate							/	Poisoning_Rate							/
		0%	1%	2%	5%	10%	15%	20%	/	0%	1%	2%	5%	10%	15%	20%	/
DOH2020 (VAE)	ECOD	0.945	0.996	0.991	0.887	0.833	0.869	0.879	0.914	0.783	0.965	0.929	0.595	0.480	0.574	0.612	0.705
	COPOD	0.973	0.999	0.998	0.893	0.950	0.948	0.955	0.959	0.872	0.992	0.984	0.611	0.803	0.802	0.849	0.845
	KDE	0.647	0.513	0.513	0.501	0.490	0.523	0.531	0.531	0.616	0.183	0.182	0.176	0.172	0.189	0.201	0.246
	OCSVM (μ=0.1)	0.999	0.999	0.997	0.941	0.887	0.892	0.879	0.942	0.999	0.999	0.999	0.986	0.971	0.972	0.968	0.985
	OCSVM (μ=0.5)	0.999	0.999	0.998	0.977	0.935	0.931	0.922	0.966	0.999	0.999	0.998	0.995	0.984	0.982	0.979	0.991
	LOF	0.954	0.456	0.397	0.393	0.368	0.587	0.549	0.529	0.646	0.158	0.144	0.144	0.138	0.198	0.185	0.230
	INNE	0.995	0.994	0.993	0.981	0.910	0.852	0.824	0.936	0.982	0.941	0.945	0.861	0.543	0.419	0.379	0.724
	IForest	0.983	0.998	0.982	0.885	0.863	0.817	0.786	0.902	0.934	0.991	0.916	0.625	0.585	0.510	0.443	0.715
	LSCP	0.969	0.979	0.986	0.891	0.888	0.899	0.901	0.930	0.802	0.906	0.923	0.569	0.549	0.636	0.656	0.720
	SUOD	0.990	0.999	0.998	0.951	0.965	0.931	0.921	0.965	0.995	0.990	0.981	0.793	0.858	0.776	0.746	0.877
	Proposed PRAD (Ours)	0.998	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.996	0.998	0.998	0.998	0.998	0.999	0.999	0.998
CTU13 (VAE)	ECOD	0.780	0.555	0.518	0.562	0.483	0.500	0.491	0.556	0.338	0.201	0.188	0.206	0.178	0.183	0.180	0.211
	COPOD	0.549	0.430	0.448	0.436	0.438	0.468	0.424	0.456	0.197	0.168	0.173	0.169	0.168	0.175	0.165	0.174
	KDE	0.659	0.539	0.558	0.564	0.569	0.575	0.587	0.579	0.619	0.317	0.347	0.400	0.357	0.379	0.345	0.395
	OCSVM (μ=0.1)	0.900	0.641	0.682	0.726	0.715	0.724	0.726	0.731	0.563	0.321	0.330	0.346	0.335	0.338	0.337	0.367
	OCSVM (μ=0.5)	0.897	0.639	0.680	0.723	0.714	0.722	0.724	0.728	0.554	0.316	0.325	0.341	0.331	0.334	0.333	0.362
	LOF	0.998	0.706	0.718	0.722	0.664	0.671	0.671	0.736	0.987	0.617	0.633	0.636	0.344	0.373	0.379	0.567
	INNE	0.986	0.909	0.881	0.712	0.649	0.639	0.627	0.772	0.926	0.564	0.494	0.463	0.364	0.394	0.360	0.509
	IForest	0.856	0.607	0.811	0.634	0.566	0.510	0.499	0.640	0.445	0.223	0.399	0.241	0.214	0.194	0.188	0.272
	LSCP	0.807	0.642	0.671	0.609	0.527	0.546	0.529	0.619	0.376	0.248	0.311	0.286	0.198	0.197	0.202	0.260
	SUOD	0.910	0.712	0.691	0.626	0.545	0.530	0.522	0.648	0.540	0.277	0.287	0.261	0.206	0.209	0.201	0.283
	Proposed PRAD (Ours)	1.0	1.0	1.0	1.0	1.0	0.765	0.765	0.933	1.0	1.0	1.0	1.0	1.0	0.892	0.892	0.969
Representation Method	**Anomaly Detection**	**AUC_ROC**							**Average AUC_ROC**	**AUC_PR**							**Average AUC_PR**
		Poisoning_Rate							/	**Poisoning_Rate**							/
		0%	**1%**	**2%**	**5%**	**10%**	**15%**	**20%**	/	**0%**	**1%**	**2%**	**5%**	**10%**	**15%**	**20%**	/
AE (DOH2021)	ECOD	0.923	0.990	0.990	0.991	0.992	0.995	0.995	0.982	0.632	0.902	0.899	0.913	0.928	0.947	0.944	0.881
	COPOD	0.950	0.993	0.989	0.994	0.996	0.998	0.998	0.988	0.689	0.925	0.893	0.934	0.958	0.971	0.975	0.906
	KDE	0.973	0.909	0.957	0.980	0.996	0.999	0.999	0.973	0.877	0.567	0.700	0.829	0.977	0.997	0.998	0.849
	OCSVM (μ=0.1)	0.984	0.999	0.999	0.993	0.962	0.940	0.929	0.972	0.928	0.985	0.978	0.938	0.846	0.784	0.741	0.886
	OCSVM (μ=0.5)	0.989	0.999	0.999	0.999	0.999	0.999	0.999	0.998	0.919	0.988	0.987	0.991	0.988	0.987	0.991	0.979
	LOF	0.999	0.644	0.643	0.623	0.609	0.591	0.567	0.668	0.999	0.273	0.274	0.257	0.241	0.224	0.205	0.353
	INNE	0.997	0.998	0.997	0.995	0.992	0.989	0.983	0.993	0.978	0.985	0.963	0.928	0.897	0.876	0.833	0.923
	IForest	0.988	0.999	0.999	0.998	0.997	0.999	0.997	0.997	0.920	0.986	0.987	0.973	0.975	0.986	0.969	0.971
	LSCP	0.941	0.967	0.966	0.980	0.986	0.988	0.986	0.973	0.864	0.752	0.746	0.829	0.881	0.897	0.887	0.837
	SUOD	0.994	0.996	0.990	0.992	0.988	0.994	0.988	0.992	0.951	0.952	0.889	0.909	0.886	0.932	0.885	0.915
	Proposed PRAD (Ours)	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.998	0.999	0.999	0.999	0.998	0.998	0.998	0.998
AE (CTU13)	ECOD	0.863	0.714	0.634	0.697	0.705	0.813	0.730	0.737	0.550	0.320	0.248	0.330	0.284	0.475	0.363	0.367
	COPOD	0.958	0.851	0.750	0.694	0.532	0.680	0.643	0.730	0.806	0.493	0.336	0.313	0.197	0.256	0.237	0.377
	KDE	0.969	0.695	0.804	0.878	0.835	0.765	0.647	0.799	0.878	0.553	0.605	0.650	0.491	0.558	0.399	0.591
	OCSVM (μ=0.1)	0.981	0.846	0.650	0.583	0.759	0.927	0.932	0.811	0.910	0.562	0.368	0.311	0.369	0.713	0.682	0.559
	OCSVM (μ=0.5)	0.975	0.909	0.883	0.855	0.647	0.786	0.771	0.832	0.869	0.567	0.654	0.600	0.247	0.466	0.422	0.546
	LOF	0.988	0.735	0.746	0.775	0.790	0.802	0.806	0.806	0.905	0.601	0.605	0.623	0.619	0.659	0.661	0.668
	INNE	0.975	0.969	0.954	0.946	0.913	0.925	0.897	0.940	0.828	0.840	0.758	0.661	0.539	0.690	0.641	0.708
	IForest	0.984	0.941	0.902	0.842	0.704	0.715	0.639	0.818	0.916	0.718	0.613	0.520	0.512	0.491	0.429	0.600
	LSCP	0.947	0.846	0.790	0.805	0.721	0.823	0.785	0.817	0.753	0.471	0.378	0.382	0.289	0.439	0.361	0.439
	SUOD	0.958	0.908	0.846	0.828	0.764	0.826	0.755	0.841	0.789	0.612	0.458	0.429	0.321	0.465	0.361	0.491
	Proposed PRAD (Ours)	1.0	1.0	1.0	1.0	1.0	1.0	0.765	0.966	1.0	1.0	1.0	1.0	1.0	1.0	0.892	0.985

Using different encoding strategies.

as the severity of poisoning attacks intensifies, particularly when the poisoning rate surpasses 10%, the anomaly detection performance on the CTU13 degrades significantly when AE or VAE is used as the FEM. Specifically, the AUCROC scores undergoes a steep decline, even falling below that of certain comparison algorithms. Additionally, the AUCPR scores also drops to 0.892. Although AE and VAE still surpass traditional anomaly detection algorithms as the intensity of data poisoning attacks increases, their overall performance falls short of that achieved by employing VRNN as the FEM, as indicated by the average results. These observations validate our choice of using VRNN as the FEM in our experiments. Despite its higher complexity compared to AE and VAE, VRNN exhibits superior experimental performance, particularly when confronted with the increasing intensity of data poisoning attacks.

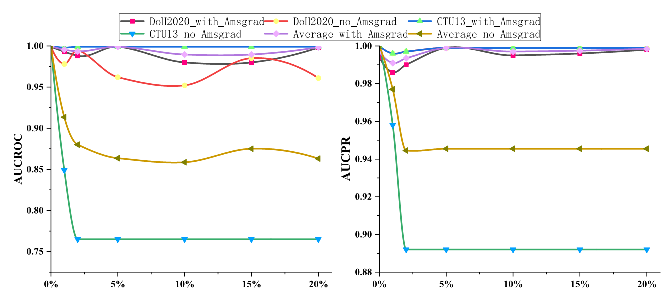

Fig. 6. The impact of Amsgrad backpropagation algorithm on anomaly detection results.

2) Effect of Amsgrad Backpropagation Algorithm on Poisoning Defense: We investigate the impact of incorporating the Amsgrad backpropagation algorithm during FEM training on the overall experimental results. Fig. 6 presents the experimental results assessed using the AUCROC and AUCPR scores. An analysis of the AUCROC scores reveals that employing the Amsgrad backpropagation algorithm during FEM training leads to noticeable variations in the resulting performance, particularly on the CTU13. When using the conventional AdamW algorithm, even a moderately intensified poisoning attack results in a significant decline in performance across the entire CTU13. In contrast, on the DoH2020, as the intensity of the poisoning attack escalates, its performance experiences a gradual decline before eventually stabilizing. It is noteworthy that the impact on the results for the CTU13 is markedly different. As the intensity of poisoning attacks increases, utilizing the conventional AdamW algorithm leads to a sharp decrease in the AUCROC and AUCPR scores on the CTU13, causing them to drop to

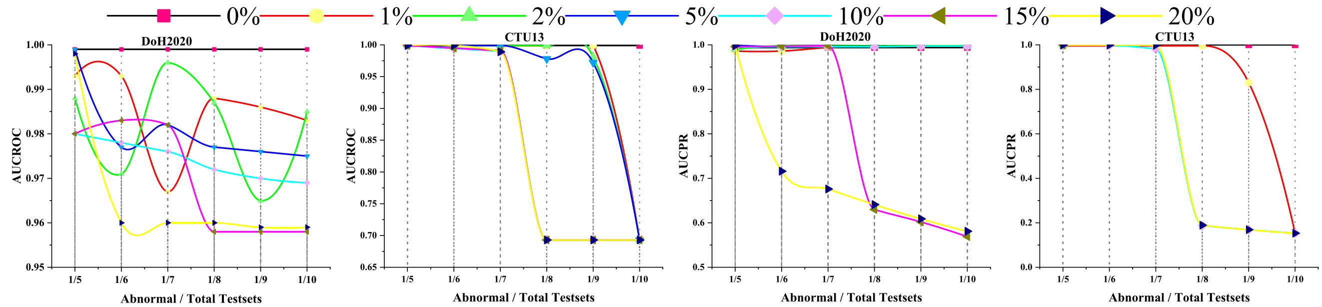

Fig. 7. The impact of poisoning attack intensity and data imbalance on anomaly detection performance: From left to right, the results of DoH2020 and CTU13 affected by different poisoning attack intensity and different data imbalance on the two indicators of AUCROC and AUCPR.

approximately 0.75. This substantial decline results in a notable increase in false positives and false negatives. In summary, integrating the Amsgrad algorithm during FEM training effectively mitigates the issues caused by poisoning attacks in subsequent anomaly detection tasks. It demonstrates significant benefits in preserving performance stability and minimizing false positives and false negatives.

3) The Joint Effect of Data Imbalance and Poisoning Attacks: We study the overall performance of PRAD in the context of data poisoning attacks and data imbalance issues. The results of these experiments are illustrated in Fig. 7. We investigate the impact of increasing data poisoning attacks and the growing problem of data imbalance. In the absence of data poisoning attacks, PRAD demonstrates robust performance in the face of data imbalance problems. However, as the intensity of data poisoning attacks increases and the proportion of abnormal data decreases, PRAD's performance on the DoH2020 dataset remains remarkably resilient. The AUCROC scores gradually decreases and stabilizes around 0.975. Meanwhile, the AUCPR scores only experiences a significant decrease when the poisoning rate reaches 20%. It is important to note that the CTU13 displays distinct characteristics. The impact of data imbalance becomes more evident as the intensity of poisoning attacks deepens. Particularly, when the poisoning rate is 20%, and the abnormal data accounts for $1/6$ of the overall test data, the effect becomes most pronounced. Similarly, the AUCPR scores indicator exhibits a more prominent impact of data imbalance as the intensity of poisoning attacks intensifies. In conclusion, when confronted with data poisoning attacks, it is crucial to ensure that the proportion of abnormal samples in the total dataset is no less than 20% when utilizing PRAD for anomaly detection.

4) Time Complexity: In this part, we present a time complexity evaluation of the proposed PRAD approach. To facilitate a comprehensive comparison, we have additionally included the training complexity analysis of competing algorithms in Table IV. In the table, n denotes the size of the input data, t represents the number of trees, and ψ represents the subspace sample size. We perform a time complexity analysis on each component of the model. Notably, since the front end of the model is based on the FEM of VRNN, we employ two indicators: Floating-point Operations (Flops) and Parameters (Params). The training Flops and Params of the proposed model are approximately 11.9 times and 13.1 times higher compared to VAE and AE, respectively. However, the predicted Params of the proposed model is 2.12 M, which is around 3.3 times that of VAE and AE. Similarly, the back end of PRAD employs a clustering ADM based on GMM. The back end's training time complexity is $\mathcal{O}(n^2)$, while the prediction process time complexity is $\mathcal{O}(n)$, indicating that the training of the model's front-end feature extraction and back-end clustering anomaly detection modules incurs the highest time complexity.

TABLE IV
TIME COMPLEXITY ANALYSIS OF PRAD AND COMPETITIVE ALGORITHMS

Front-End Model				Backend Algorithm	
Model	**Flops (MMAC)**	**Param (M)**	**Params Test(M)**	**Anomaly Detection**	**Complexity**
VRNN	16.9	16.88	2.12	**ECOD**	$\mathcal{O}(n)$
				COPOD	$\mathcal{O}(n)$
				KDE	$\mathcal{O}(n^2)$
				OCSVM	$\mathcal{O}(n^2)$
VAE	1.42	1.42	0.64	**LOF**	$\mathcal{O}(n^2)$
				INNE	$\mathcal{O}(t\psi^2)$
				IForest	$\mathcal{O}(t\psi\log\psi)$
AE	1.29	1.29	0.64	**LSCP**	$\mathcal{O}(n\log n)$
				SUOD	$\mathcal{O}(n)$
				Proposed PRAD	$\mathcal{O}(n^2)$

VI. CONCLUSION

In this paper, we proposed PRAD, a semi-supervised encrypted traffic anomaly detection framework under poisoning attack scenarios. Specifically, to solve the inter-class reduction problem caused by poisoning attacks, we first propose a feature encoding module based on VRNN variants, and then adopt the Amsgrad gradient descent algorithm and warm-up strategy to alleviate this problem. In addition, FAM is also introduced to evaluate the impact of poisoning attacks on inter-class distances and analyze the distribution of reconstruction errors, providing important prior information for subsequent anomaly detection tasks. Finally, an online clustering algorithm is designed to effectively detect anomalies in encrypted traffic, which utilizes the extracted features and their associated reconstruction errors to effectively identify anomalies in encrypted traffic. Extensive experiments have verified that PRAD can cope with anomaly detection of encrypted traffic in poisoning attack scenarios. In future work, we will address the issue of backdoor attacks in encrypted traffic.

References

[1] Z. Durumeric et al., "The security impact of HTTPS interception," in *Proc. 24th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2017, pp. 1–14.

[2] G. D. L. T. Parra, P. Rad, and K.-K. R. Choo, "Implementation of deep packet inspection in smart grids and industrial Internet of Things: Challenges and opportunities," *J. Netw. Comput. Appl.*, vol. 135, pp. 32–46, 2019.

[3] J. Sherry, C. Lan, R. A. Popa, and S. Ratnasamy, "BlindBox: Deep packet inspection over encrypted traffic," in *Proc. ACM Conf. Special Int. Group Data Commun.*, 2015, pp. 213–226.

[4] H. Gao, B. Qiu, R. J. D. Barroso, W. Hussain, Y. Xu, and X. Wang, "TSMAE: A novel anomaly detection approach for Internet of Things time series data using memory-augmented autoencoder," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2978–2990, Sep./Oct. 2022.

[5] J. Ahmed, H. H. Gharakheili, C. Russell, and V. Sivaraman, "Automatic detection of DGA-enabled malware using SDN and traffic behavioral modeling," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 2922–2939, Jul./Aug. 2022.

[6] Z. Li, Y. Zhao, X. Hu, N. Botta, C. Ionescu, and G. Chen, "ECOD: Unsupervised outlier detection using empirical cumulative distribution functions," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12181–12193, Dec. 2023.

[7] Z. Li, Y. Zhao, N. Botta, C. Ionescu, and X. Hu, "COPOD: Copula-based outlier detection," in *Proc. IEEE Int. Conf. Data Mining*, 2020, pp. 1118–1123.

[8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 1, pp. 1–39, 2012.

[9] Y. Zhao, Z. Nasrullah, M. K. Hryniewicki, and Z. Li, "LSCP: Locally selective combination in parallel outlier ensembles," in *Proc. SIAM Int. Conf. Data Mining*, 2019, pp. 585–593.

[10] Y. Zhao et al., "SUOD: Accelerating large-scale unsupervised heterogeneous outlier detection," *Proc. Mach. Learn. Syst.*, vol. 3, pp. 463–478, 2021.

[11] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "KITSUNE: An ensemble of autoencoders for online network intrusion detection," in *Proc. 25th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.

[12] V. L. Cao, M. Nicolau, and J. McDermott, "Learning neural representations for network anomaly detection," *IEEE Trans. Cybern.*, vol. 49, no. 8, pp. 3074–3087, Aug. 2019.

[13] G. Bovenzi, G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapé, "Network anomaly detection methods in IoT environments via deep learning: A fair comparison of performance and robustness," *Comput. Secur.*, vol. 128, pp. 103–167, 2023.

[14] I. Diakonikolas, G. Kamath, D. Kane, J. Li, J. Steinhardt, and A. Stewart, "Sever: A robust meta-algorithm for stochastic optimization," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 1596–1606.

[15] J. Steinhardt, P. W. W. Koh, and P. S. Liang, "Certified defenses for data poisoning attacks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 3517–3529.

[16] H. Lu, T. Wang, X. Xu, and T. Wang, "Cognitive memory-guided autoencoder for effective intrusion detection in Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3358–3366, May 2022.

[17] J. Ashraf, A. D. Bakhshi, N. Moustafa, H. Khurshid, A. Javed, and A. Beheshti, "Novel deep learning-enabled LSTM autoencoder architecture for discovering anomalous events from intelligent transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4507–4518, Jul. 2021.

[18] M. Barreno, B. Nelson, R. Sears, A. D. Joseph, and J. D. Tygar, "Can machine learning be secure?," in *Proc. ACM Symp. Inf., Comput. Commun. Secur.*, 2006, pp. 16–25.

[19] Y. Hua, F. Wan, H. Gan, Y. Zhang, and X. Qing, "Distributed estimation with cross-verification under false data-injection attacks," *IEEE Trans. Cybern.*, vol. 53, no. 9, pp. 5840–5853, Sep. 2023.

[20] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to {Byzantine-Robust} federated learning," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 1605–1622.

[21] P. Zhao et al., "Garbage in, garbage out: Poisoning attacks disguised with plausible mobility in data aggregation," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 3, pp. 2679–2693, Jul.–Sep. 2021.

[22] Y. Liu et al., "Trojaning attack on neural networks," in *Proc. 25th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–17.

[23] A. Shafahi et al., "Poison frogs! targeted clean-label poisoning attacks on neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 151–178.

[24] W. R. Huang, J. Geiping, L. Fowl, G. Taylor, and T. Goldstein, "MetaPoison: Practical general-purpose clean-label data poisoning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 12080–12091.

[25] M. Al-Hawawreh, N. Moustafa, S. Garg, and M. S. Hossain, "Deep learning-enabled threat intelligence scheme in the Internet of Things networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 4, pp. 2968–2981, Oct.–Dec. 2021.

[26] P. W. Koh, J. Steinhardt, and P. Liang, "Stronger data poisoning attacks break data sanitization defenses," *Mach. Learn.*, vol. 111, no. 1, pp. 1–47, 2022.

[27] F. Suya, S. Mahloujifar, A. Suri, D. Evans, and Y. Tian, "Model-targeted poisoning attacks with provable convergence," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 10000–10010.

[28] C. Zhu, W. R. Huang, H. Li, G. Taylor, C. Studer, and T. Goldstein, "Transferable clean-label poisoning attacks on deep neural nets," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7614–7623.

[29] S. Li et al., "Hidden backdoors in human-centric language models," in *Proc. Conf. Comput. Commun. Secur.*, 2021, pp. 3123–3140.

[30] G. Bovenzi, A. Foggia, S. Santella, A. Testa, V. Persico, and A. Pescapé, "Data poisoning attacks against autoencoder-based anomaly detection models: A robustness analysis," in *Proc. IEEE Int. Conf. Commun.*, 2022, pp. 5427–5432.

[31] Y. Li, J. Yang, Y. Song, L. Cao, J. Luo, and L.-J. Li, "Learning from noisy labels with distillation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 1910–1918.

[32] E. Malach and S. Shalev-Shwartz, "Decoupling" when to update" from" how to update"," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 961–971.

[33] N. Peri et al., "Deep *k*-NN defense against clean-label data poisoning attacks," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 55–70.

[34] R. Taheri, R. Javidan, M. Shojafar, Z. Pooranian, A. Miri, and M. Conti, "On defending against label flipping attacks on malware detection systems," *Neural Comput. Appl.*, vol. 32, pp. 14781–14800, 2020.

[35] M. Jagielski, A. Oprea, B. Biggio, C. Liu, C. Nita-Rotaru, and B. Li, "Manipulating machine learning: Poisoning attacks and countermeasures for regression learning," in *Proc. IEEE Symp. Secur. Privacy*, 2018, pp. 19–35.

[36] Z. Tian, L. Cui, J. Liang, and S. Yu, "A comprehensive survey on poisoning attacks and countermeasures in machine learning," *ACM Comput. Surv.*, vol. 55, no. 8, pp. 1–35, 2022.

[37] J. Chen, X. Zhang, R. Zhang, C. Wang, and L. Liu, "De-Pois: An attack-agnostic defense against data poisoning attacks," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 3412–3425, 2021.

[38] J. Chung, K. Kastner, L. Dinh, K. Goel, A. C. Courville, and Y. Bengio, "A recurrent latent variable model for sequential data," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 2980–2988.

[39] N. Megiddo and K. J. Supowit, "On the complexity of some common geometric location problems," *SIAM J. Comput.*, vol. 13, no. 1, pp. 182–196, 1984.

[40] K. Pillutla, S. M. Kakade, and Z. Harchaoui, "Robust aggregation for federated learning," *IEEE Trans. Signal Process.*, vol. 70, pp. 1142–1154, 2022.

[41] M. Rashid and J. A. Nanzer, "Online expectation-maximization based frequency and phase consensus in distributed phased arrays," *IEEE Trans. Commun.*, vol. 71, no. 6, pp. 3721–3735, Jun. 2023.

[42] S. Garcia, M. Grill, J. Stiborek, and A. Zunino, "An empirical comparison of botnet detection methods," *Comput. Secur.*, vol. 45, pp. 100–123, 2014.

[43] M. MontazeriShatoori, L. Davidson, G. Kaur, and A. H. Lashkari, "Detection of DoH tunnels using time-series classification of encrypted traffic," in *Proc. IEEE Int. Conf. Dependable, Autonomic Secure Comput., Int. Conf. Pervasive Intell. Comput., Int. Conf. Cloud Big Data Comput., Int. Conf. Cyber Sci. Technol. Congr.*, 2020, pp. 63–70.

[44] W. Hu, J. Gao, B. Li, O. Wu, J. Du, and S. Maybank, "Anomaly detection using local kernel density estimation and context-based regression," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 2, pp. 218–233, Feb. 2020.

[45] T. R. Bandaragoda, K. M. Ting, D. Albrecht, F. T. Liu, Y. Zhu, and J. R. Wells, "Isolation-based anomaly detection using nearest-neighbor ensembles," *Comput. Intell.*, vol. 34, no. 4, pp. 968–998, 2018.

Zhangfa Wu received the M.S. degree in electronic science and technology in 2020 from the School of Electronic Information Engineering, Northwestern Polytechnical University, Xi'an, China, where he is currently working toward the Ph.D. degree in electronic science and technology with the School of Electronic Information Engineering. His research interests include anomaly detection, network security, deep learning, signal processing, and image processing.

Huifang Li received the Ph.D degree from Northwestern Polytechnical University, Xi'an, China, in 2004. From 2010 to 2011, he was with the University of North Carolina, Chapel Hill, NC, USA, as a Visiting Scholar sponsored by the China Scholarship Council. He is currently a Professor with the School of Electronic Information, Northwestern Polytechnical University. His research interests include deep learning, network security, image processing, quantum information processing, intelligent information processing, multimedia information processing, and measurement and control technology.

Yekui Qian was born in 1980 in Anqing, Anhui, China. He received the Ph.D. degree from People's Liberation Army University of Science and Technology, Beijing, China. He is currently a Professor with People's Liberation Army Air Force Academy, Zhengzhou, China. His research interests include network measurement, vulnerability mining, deep learning, and network security.

Yi Hua received the M.S. degree in signal and information processing from the School of Electronic and Information Engineering, Southwest University, Chongqing, China, in 2020. He is currently working toward the Ph.D. degree in aerospace science and technology with the School of Aeronautics, Northwestern Polytechnical University, Xi'an, China. From 2023, he is with the School of Electrical Engineering and Computer Science, The University of Queensland, Brisbane, QLD, Australia, as a joint training doctoral student under CSC scholarship. His research interests include distributed signal processing, network security, image processing, robots, path planning, and information fusion of multiple aircraft.

Hongping Gan (Member, IEEE) received the Ph.D degree in communication and information engineering from the State Key Laboratory of ISN, Xidian University, Xi'an, China, in 2020. From 2018 to 2019, he was with the School of Information Technology and Electrical Engineering, The University of Queensland, Brisbane, QLD, Australia, as a Visiting Scholar sponsored by the China Scholarship Council. He is currently an Associate Professor with the School of Software, Northwestern Polytechnical University, Xi'an. His research interests include deep learning, compressive sensing, image processing, PolSAR ship detection, and network security.